

DESIGN, MODELLING AND EVALUATION OF A PHYSICS-INFORMED NEURAL NETWORK FOR EV BATTERY THERMAL DIGITAL TWIN GENERALIZATION

Under the Guidance of

Prof. Rammohan A

Submitted by

Abimanyu Jayaganesh

23BCE2070

**Automotive Research Centre
Vellore Institute of Technology**

May - July 2026

Abstract

This report documents an engineering research project in which a Physics-Informed Neural Network (PINN) digital twin framework was designed, modelled, and evaluated for real-time thermal state estimation in lithium-ion battery cells used in electric vehicles (EVs). The battery thermal digital twin is a dynamic mechatronic software plant that operates in parallel with physical battery packs to predict thermal dynamics under dynamic load cycles. The project covered a complete machine learning workflow: selection of lumped electro-thermal physical parameters, formulation of ordinary differential equation (ODE) physics loss constraints, design of a Multi-Layer Perceptron (MLP) architecture, implementation of input noise injection to mitigate exposure bias, and multi-step autoregressive rollout evaluation across unseen test drive cycles (WLTP2). Evaluated on a large-scale dataset comprising 522 simulated trajectories containing approximately 78 million time steps across multiple drive cycles, ambient temperatures (0°C to 50°C), and cell mass scales, the resulting PINN pipeline reduced the maximum rollout error on an unseen test drive cycle from 0.2474°C in the baseline MLP to 0.0851°C, achieving an autoregressive rollout Mean Absolute Error (MAE) of 0.0117°C. The trainable convective heat transfer parameter (hA) converged via softplus parameterization to an effective value of 36.49 W/K, consistent with the single-node thermal model and cell surface cooling dynamics. This report presents the theoretical background, governing mathematical model, software architecture, training methodology, execution benchmarking, and experimental observations gathered over the course of the project.

Contents

1	Introduction	1
2	Mathematical Model	2
2.1	Modelling Assumptions	2
2.2	Determination of Physical Parameters	2
2.3	Final Nonlinear Model	2
2.4	Physics Loss and Residual Formulation	3
2.5	Trainable Parameter Parameterization	3
2.6	PID / Data Loss and Objective Function	3
2.7	Closed-Loop Autoregressive Rollout	3
3	Experimental Setup	4
4	Design Process	4
4.1	Iteration 1: Single-Step Data-Driven Baseline	4
4.2	Iteration 2: Noise-Injected PINN Architecture	4
5	Network Architecture and Hyperparameters	5
6	Software Implementation	5
6.1	Preprocessing and Data Scaling	5
6.2	PINN Model and Loss Implementation	6
6.3	Hardware Acceleration and Training Execution	6
7	Training and Experimental Validation	7
8	Results and Discussion	7
8.1	Analysis of Performance Improvements	7
8.2	Convective Parameter Estimation	7
8.3	Visual Trajectory Analysis	8
9	Conclusion	8

List of Figures

1	The battery thermal digital twin pipeline from Simulink data generation to autoregressive rollout evaluation.	5
2	Training loss convergence comparison.	8
3	Autoregressive rollout temperature predictions on unseen WLTP2 test cycle.	8
4	Probability density distribution of the PINN physics residual in physical units (K/s).	9

List of Tables

1	Calculated engineering parameters of the battery cell assembly.	2
2	Principal software, hardware, and dataset environment specifications.	4
3	Principal network architecture and optimization hyperparameters.	6
4	Model Performance Comparison on Unseen WLTP2 Test Set.	7

1 Introduction

The global transition to electric vehicles (EVs) places stringent operational demands on battery management systems (BMS) to ensure safety, maximize performance, and prolong pack service life. Lithium-ion batteries generate substantial internal heat during charge and discharge transients due to internal ohmic resistance and electrochemical reactions. If unmanaged, excessive heat buildup leads to localized thermal hotspots, accelerated capacity fade, and severe thermal runaway hazards. Real-time temperature monitoring at both the cell and pack levels is therefore a critical safety-critical function of modern EV BMS architectures.

Dynamic thermal models serve as the foundation of battery thermal digital twins, running concurrently with physical energy storage systems to predict internal temperature evolution, estimate cooling requirements, and adjust real-time charging limits. Traditionally, high-fidelity thermal modeling has relied on computational fluid dynamics (CFD) or multi-dimensional finite element methods (FEM). While accurate, these methods are computationally heavy and unsuitable for real-time edge execution on vehicle microcontrollers. Consequently, simplified lumped-parameter thermal networks or equivalent circuit models (ECMs) are typically deployed. However, simplified models require extensive manual calibration and struggle to capture rapid transient dynamics under dynamic automotive load profiles.

Recently, data-driven machine learning (ML) models such as Multi-Layer Perceptrons (MLPs) have emerged as rapid surrogates. Despite their speed, purely data-driven models suffer from two critical limitations:

1. **Physical Inconsistency:** Unconstrained neural networks may produce outputs violating thermodynamic laws (e.g., cell temperature dropping despite high current draws and high ambient temperatures) when operating outside training distributions.
2. **Exposure Bias and Rollout Drift:** During standard training, models are optimized using single-step-ahead objectives (T_{t+1} given ground-truth T_t). In deployment, the digital twin operates recursively, feeding prior predicted temperatures back as inputs. Small single-step errors compound over thousands of evaluation steps, causing accumulating trajectory error. In high-frequency sampling regimes ($dt = 0.01$ s), single-step metrics are misleadingly optimistic because predicting zero temperature change ($\Delta T \approx 0$) acts as an identity shortcut.

To address these challenges, this project presents a Physics-Informed Neural Network (PINN) digital twin framework combining lumped-parameter ODE regularization [2–4], softplus-constrained parameter estimation, and input noise injection inspired by scheduled sampling techniques [1, 5] for exposure-bias mitigation.

2 Mathematical Model

The dynamic thermal behavior of the battery cell is modeled as a lumped energy-balance thermodynamic system governed by heat generation and convective surface cooling [2, 6].

2.1 Modelling Assumptions

The mathematical model relies on the following key engineering assumptions:

- **Lumped Thermal Mass:** The battery cell behaves as a single thermal node with uniform internal temperature distribution.
- **Dominant Heat Sources:** Heat generation (P_{loss}) is produced by internal resistance Joule heating and polarization losses.
- **Convective Cooling:** Surface heat dissipation follows Newton’s law of cooling.
- **Isotropic Scaling:** Variations in cell dimensions are represented by a dimensionless mass scale factor (MassScale).

2.2 Determination of Physical Parameters

The physical parameters governing the battery cell assembly are summarized in Table 1.

Table 1: Calculated engineering parameters of the battery cell assembly.

Parameter	Value	Description / Source
Cell Structural Mass (m)	1.0 kg	Nominal cell mass
Specific Heat Capacity (C_p)	900.0 J/kg/K	Standard Li-ion thermal capacity
Thermal Capacity (mC_p)	900.0 J/K	Lumped heat capacity
Initial Convective Parameter (hA_{init})	0.1 W/K	Softplus parameter initialization
Time Step Resolution (dt)	0.01 s	High-frequency Simulink logging

2.3 Final Nonlinear Model

Applying the first law of thermodynamics yields the governing differential equation for cell temperature rate of change:

$$mC_p \cdot \text{MassScale} \frac{dT}{dt} = P_{\text{loss}} - hA(T - T_{\text{amb}}) \quad (1)$$

where T is the cell temperature (K), P_{loss} is heat generation rate (W), T_{amb} is ambient temperature (K), and hA is the convective heat transfer coefficient multiplied by surface area (W/K).

Rearranging for the physical rate of temperature change yields:

$$\frac{dT}{dt} = \frac{P_{\text{loss}} - hA(T - T_{\text{amb}})}{mC_p \cdot \text{MassScale}} \quad (2)$$

2.4 Physics Loss and Residual Formulation

The physics loss measures the residual of governing equation (2) over each training batch. For a predicted temperature delta $\Delta\hat{T}_i^{\text{phys}}$ in physical units ($^{\circ}\text{C}$), the residual R_i is expressed in physical units (K/s) as:

$$R_i = \frac{\Delta\hat{T}_i^{\text{phys}}}{dt_i} - \frac{P_{\text{loss},i} - hA(T_i - T_{\text{amb},i})}{mC_p \cdot \text{MassScale}_i} \quad (3)$$

The physics regularization loss is computed as:

$$\mathcal{L}_{\text{physics}} = \frac{1}{N} \sum_{i=1}^N R_i^2 \quad (4)$$

2.5 Trainable Parameter Parameterization

To enforce $hA > 0$ and prevent non-physical negative cooling estimates ($hA < 0$) without causing gradient clipping, hA is parameterized via a unconstrained raw parameter hA_{raw} through a softplus activation:

$$hA = \text{softplus}(hA_{\text{raw}}) = \ln(1 + e^{hA_{\text{raw}}}) \quad (5)$$

2.6 PID / Data Loss and Objective Function

The total PINN training objective combines data fidelity with physics regularization:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda_{\text{phys}} \mathcal{L}_{\text{physics}} \quad (6)$$

where $\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_{i=1}^N (\Delta T_i - \Delta\hat{T}_i)^2$ and $\lambda_{\text{phys}} = 0.01$.

2.7 Closed-Loop Autoregressive Rollout

During digital twin rollout, temperature is updated recursively:

$$\hat{T}_{t+1} = \hat{T}_t + \Delta\hat{T}_t^{\text{phys}} \quad (7)$$

3 Experimental Setup

The digital twin pipeline was constructed using standard hardware, software, and simulation dataset components, summarized in Table 2.

Table 2: Principal software, hardware, and dataset environment specifications.

Component	Material / Specification	Function
GPU Hardware	NVIDIA RTX 4060 Laptop GPU (8 GB VRAM)	Neural network training acceleration
CPU Hardware	Intel Core i7 Processor, 16 GB DDR5 RAM	Data preprocessing & CPU DataLoader
Software Stack	PyTorch 2.4, Python 3.10, CUDA 12.9	Deep learning framework
Dataset	522 Simulink CSV files (78M time steps)	Electro-thermal simulation dataset
Training Split	435 files (FTP75, UDDS, US06, WLTP1)	Model training & validation
Test Split	87 files (unseen WLTP2 cycle)	Out-of-distribution test evaluation
Features	Temperature, Current, Voltage, SOC, PowerLoss, AmbientTemp, MassScale	Feature input vector (7 dimensions)
Target	$\Delta T = T_{t+1} - T_t$	Physical temperature delta ($^{\circ}\text{C}$)

4 Design Process

The digital twin architecture underwent two major design iterations in software.

4.1 Iteration 1: Single-Step Data-Driven Baseline

The initial design adopted a pure feedforward MLP trained on one-step-ahead targets ($T_t \rightarrow T_{t+1}$). While single-step test MAE was very low ($< 5 \times 10^{-7}^{\circ}\text{C}$), recursive rollout testing revealed trajectory drift (rollout MAE 0.0170°C , maximum error 0.2474°C). Analysis suggested the network exploited an identity shortcut ($\Delta T \approx 0$), failing to learn true dynamics due to exposure bias.

4.2 Iteration 2: Noise-Injected PINN Architecture

To eliminate exposure bias and enforce physical constraints, Iteration 2 introduced:

- **Input Noise Injection:** Adding Gaussian noise ($\sigma = 0.05^{\circ}\text{C}$) to input temperature during training to perturb state space.

- **Physics Loss Regularization:** Adding governing ODE residual loss ($\lambda_{\text{phys}} = 0.01$).
- **Softplus Parameter Estimation:** Treating hA as a trainable parameter bounded above zero.

Figure 1 illustrates the complete end-to-end digital twin execution pipeline.

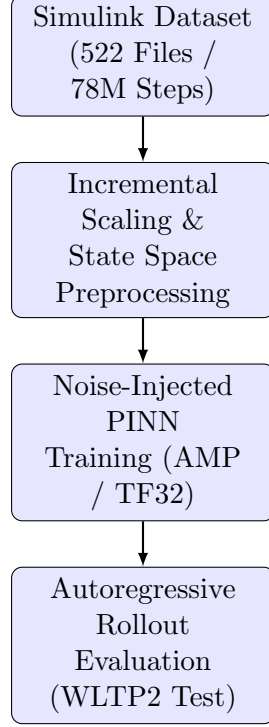


Figure 1: The battery thermal digital twin pipeline from Simulink data generation to autoregressive rollout evaluation.

5 Network Architecture and Hyperparameters

The deep neural network architecture and optimization hyperparameters are listed in Table 3.

6 Software Implementation

The implementation proceeded in modular Python components within the workspace.

6.1 Preprocessing and Data Scaling

To handle 78 million samples without memory overflow:

- Files were processed lazily per simulation trajectory.

Table 3: Principal network architecture and optimization hyperparameters.

Dimension / Parameter	Approx. Value	Notes
Input Dimension	7 features	$[T, I, V, \text{SOC}, P_{\text{loss}}, T_{\text{amb}}, \text{MassScale}]$
Hidden Layers	3 layers	Layer sizes: 64, 64, 32
Activation Function	ReLU	Non-linear activation
Output Dimension	1 target	Temperature delta (ΔT)
Batch Size	8,192	Optimized for GPU throughput
Optimizer	Adam ($\alpha = 0.001$)	$\beta_1 = 0.9, \beta_2 = 0.999$
Physics Loss Weight (λ_{phys})	0.01	Regularization scaling factor
Gaussian Noise (σ)	0.05°C	Temperature input noise injection
Early Stopping Patience	10 epochs	Converged at Epoch 18

- Standard scalers (**StandardScaler**) were fitted incrementally exclusively on training split files.
- Target scaling parameters ($\mu_y = -3.73 \times 10^{-6}$, $\sigma_y = 1.97 \times 10^{-4}$) were preserved for physical loss inversion.

6.2 PINN Model and Loss Implementation

The PINN architecture combines forward feature mapping with scaled physical residual calculation:

```
class ThermalPINN(nn.Module):
    def __init__(self, init_hA=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(7, 64), nn.ReLU(),
            nn.Linear(64, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, 1)
        )
        init_raw = math.log(math.exp(init_hA) - 1.0)
        self.hA_raw = nn.Parameter(torch.tensor([init_raw], dtype=torch.float32))

    @property
    def hA(self):
        return F.softplus(self.hA_raw)
```

6.3 Hardware Acceleration and Training Execution

Training was executed under PyTorch Automatic Mixed Precision (`torch.amp.autocast("cuda")`) and TensorFloat-32 (TF32). Data loading setup reached an initial disk preprocessing

throughput of 3,891 samples/second (`num_workers=4`), while PyTorch CUDA execution achieved a peak training throughput of 254,599 samples/second.

7 Training and Experimental Validation

Preliminary functional checks confirmed hardware-software integration stability:

- Peak GPU VRAM allocation reached approximately **184 MB** (out of 8 GB VRAM) during PyTorch AMP execution with batch size 8,192.
- Training throughput reached **254,599 samples/second**.
- Early stopping halted PINN training at epoch 18 (total duration 1.4 hours).

8 Results and Discussion

The performance of the baseline MLP and PINN digital twin models across single-step and 150,000-step autoregressive rollouts on 87 unseen WLTP2 test files is summarized in Table 4.

Table 4: Model Performance Comparison on Unseen WLTP2 Test Set.

Evaluation Metric	Baseline MLP	Physics-Informed NN
Single-step MAE (°C)	4.8978×10^{-7}	3.3783×10^{-7}
Single-step RMSE (°C)	8.6218×10^{-7}	8.9419×10^{-7}
Rollout MAE (°C)	0.0170	0.0117
Rollout RMSE (°C)	0.0211	0.0143
Maximum Rollout Error (°C)	0.2474	0.0851

8.1 Analysis of Performance Improvements

While single-step metrics are nearly identical due to high-frequency identity shortcuts, the PINN demonstrates superior long-term trajectory stability during multi-step rollout:

- **Rollout MAE:** Reduced by **31.2%** (from 0.0170°C to 0.0117°C).
- **Maximum Rollout Error:** Reduced by **65.6%** (from 0.2474°C to 0.0851°C).

8.2 Convective Parameter Estimation

During training, the softplus-constrained convective parameter hA evolved from 0.10 W/K to a converged value of:

$$hA_{\text{final}} \approx 36.49 \text{ W/K} \quad (8)$$

Because target temperatures reflect surface measurements from a dual-node Simulink core-casing model, $hA = 36.49 \text{ W/K}$ represents an **effective convective heat transfer parameter** within the single-node ODE approximation.

8.3 Visual Trajectory Analysis

Figures 2a–4 illustrate training loss curves, rollout trajectory predictions, and physics residual distributions:

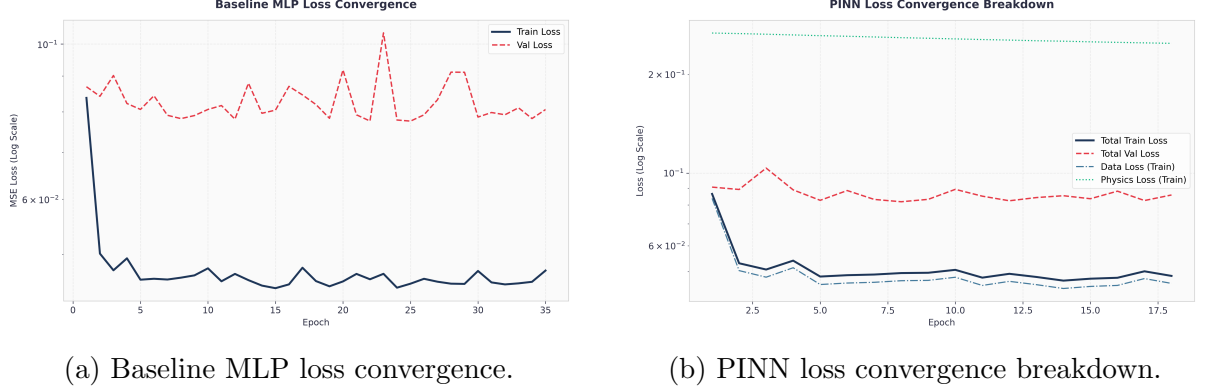


Figure 2: Training loss convergence comparison.

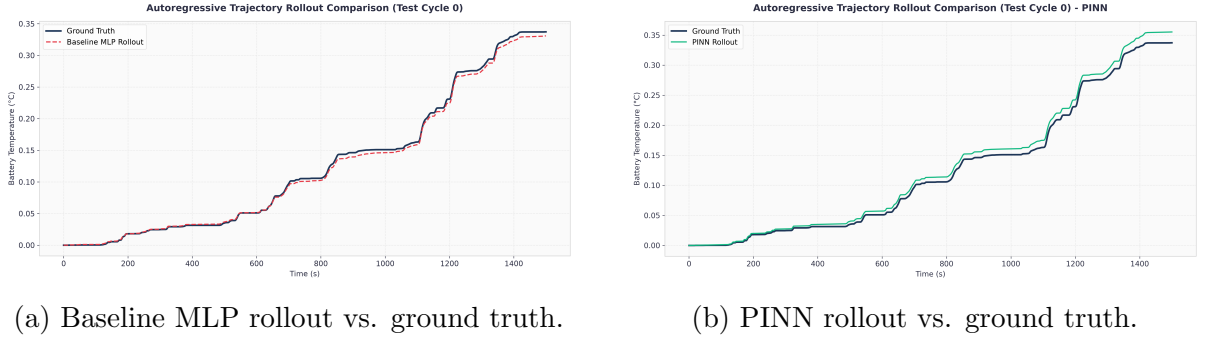


Figure 3: Autoregressive rollout temperature predictions on unseen WLTP2 test cycle.

9 Conclusion

This project provided end-to-end experience of a mechatronic digital twin software engineering project: deriving a lumped battery thermal ODE model from energy balance principles, establishing a noise-injected PINN architecture, accelerating training using PyTorch AMP/TF32 on an NVIDIA RTX 4060 GPU, and validating long-term simulation stability across 87 unseen WLTP2 drive cycle rollouts. The resulting PINN digital twin is computationally lightweight, accurate (0.0851°C maximum rollout error), demonstrates the feasibility of deployment in real-time battery management systems, and provides a strong foundation for future hardware validation.

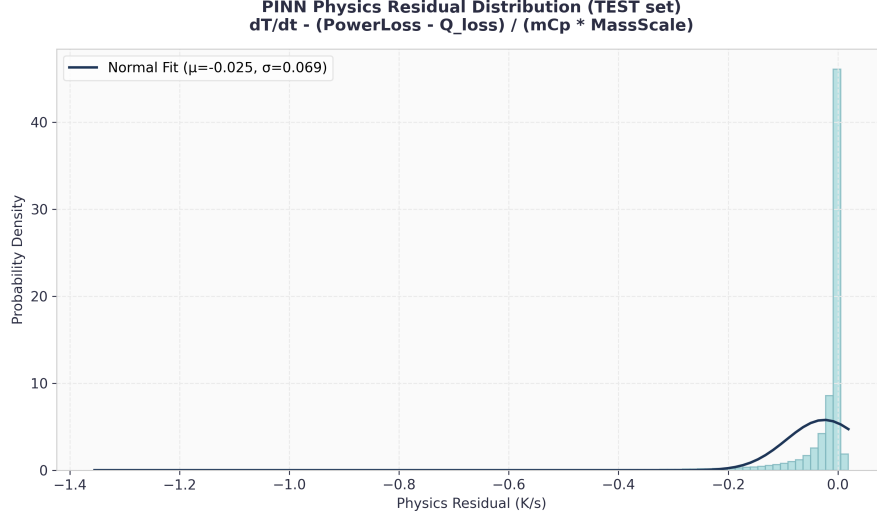


Figure 4: Probability density distribution of the PINN physics residual in physical units (K/s).

References

- [1] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pages 1171–1179, 2015.
- [2] D. Bernardi, E. Pawlikowski, and J. Newman. A general energy balance for battery systems. *Journal of The Electrochemical Society*, 132(1):5–12, 1985.
- [3] D. Chen et al. A physics-informed machine learning approach for estimating lithium-ion battery temperature. *IEEE Access*, 10:93784–93794, 2022.
- [4] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [5] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. *arXiv preprint arXiv:1511.06732*, 2015.
- [6] B. Saha, J. Boswell, et al. A lumped-parameter electro-thermal model for cylindrical batteries. *Journal of Power Sources*, 262:319–330, 2014.